

## DEVOPS MASTER COURSE

# Volume 11C

---

## Grafana, Centralized Logging, Loki, Elastic and OpenSearch

*Dashboards - Provisioning - Log Pipelines - Grafana Alloy - LogQL - Data Streams - Lifecycle - Security - Operations*

**Chapters 21-30 | 2026 Edition**

**Course Standard** Build, test, fail, diagnose, recover, document, and explain every component. A dashboard that looks attractive but cannot guide diagnosis is not production-ready.

**Beginner to Production-Ready DevOps and SRE Engineering**

## About This Part

Volume 11C turns metrics and alerting into an operator-facing investigation environment. It teaches Grafana as a governed observability interface, then builds two major centralized logging patterns: Loki for label-indexed, cost-conscious log storage and Elasticsearch/OpenSearch for full-text search, enrichment, data lifecycle, and broad analytics.

The objective is not to memorize product menus. The objective is to design reliable telemetry contracts, collection pipelines, schemas, retention controls, access boundaries, dashboards, query patterns, and failure-recovery procedures that remain useful during real incidents.

**2026 Collection Standard** Promtail reached end of life on March 2, 2026. New Loki collection laboratories in this book use Grafana Alloy. Promtail is discussed only for migration and legacy-system recognition.

## Volume 11 Roadmap

| Part | Primary Scope                                                         | Status    |
|------|-----------------------------------------------------------------------|-----------|
| 11A  | Observability foundations, Linux monitoring, telemetry, SLIs and SLOs | Completed |
| 11B  | Prometheus, PromQL, exporters, rules and Alertmanager                 | Completed |
| 11C  | Grafana, Loki, Elastic/OpenSearch and centralized logging             | This book |
| 11D  | OpenTelemetry, tracing, APM, profiling and eBPF                       | Next      |
| 11E  | SRE operations, incidents, capacity, reliability and governance       | Planned   |
| 11F  | Enterprise projects, certifications, interviews and capstone          | Planned   |

## Scope, Safety, Privacy and Cost Controls

- Use synthetic logs, test namespaces, disposable storage, and non-production notification channels for learning exercises.
- Do not place passwords, API keys, session tokens, authentication headers, payment data, health data, or personal identifiers in logs.
- Apply redaction at the application or collection layer before data reaches centralized storage; deletion after indexing is not a substitute for prevention.
- Define daily ingest budgets, retention periods, query limits, shard or stream limits, and cleanup procedures before load testing.
- Use low-cardinality Loki labels. Put high-cardinality searchable context in structured metadata or the log body rather than stream labels.
- Pin and validate configuration, dashboards, data-source definitions, pipeline files, index templates, and lifecycle policies in version control.
- Test role mappings, tenant isolation, data-source permissions, folder permissions, and audit trails before granting broad access.
- Confirm current product licensing, feature availability, migration guidance, and support status in official documentation before production adoption.

## Part C Learning Outcomes

- Deploy and govern Grafana with provisioned data sources, folders, dashboards, authentication, access controls, backups, and self-monitoring.

- Design dashboards that support overview, diagnosis, change correlation, ownership, and drill-down rather than decorative visualization.
- Create structured application logs with stable fields, severity conventions, correlation identifiers, timestamps, and privacy controls.
- Design collectors and pipelines for files, systemd, containers, Kubernetes, network endpoints, parsing, enrichment, buffering, retry, and backpressure.
- Deploy Loki, choose appropriate deployment modes, control label cardinality, query with LogQL, and operate retention and multi-tenancy.
- Use Grafana Alloy to collect, process, redact, relabel, and deliver logs to Loki with observable pipeline behavior.
- Design Elasticsearch and OpenSearch log platforms using data streams or indices, mappings, schemas, ingest pipelines, lifecycle policies, security, and dashboards.
- Build a unified observability capstone that correlates metrics, logs, alerts, deployments, and runbooks during controlled incidents.

## Table of Contents

1. Chapter 21: Grafana Architecture, Data Sources, Security and Provisioning
2. Chapter 22: Production Dashboard Engineering and Operational UX
3. Chapter 23: Grafana Alerting, Annotations, Correlations and Governance
4. Chapter 24: Centralized Logging Architecture and Structured Log Engineering
5. Chapter 25: Grafana Loki Architecture, Storage, Tenancy and Operations
6. Chapter 26: LogQL Query Engineering, Metrics from Logs and Alerting
7. Chapter 27: Grafana Alloy Log Collection for Linux, Containers and Kubernetes
8. Chapter 28: Elasticsearch Logging: ECS, Data Streams, Ingest Pipelines and Lifecycle
9. Chapter 29: OpenSearch Logging: Data Prepper, PPL, ISM, Security and Operations
10. Chapter 30: Unified Observability and Centralized Logging Platform Capstone

## Chapter 21: Grafana Architecture, Data Sources, Security and Provisioning

*Operating Grafana as a governed observability interface rather than a shared admin account*

### The Role of Grafana

Grafana sits between users and telemetry backends. It does not replace Prometheus, Loki, Elasticsearch, OpenSearch, cloud monitoring systems, or SQL databases. It provides a common query and visualization experience, permission model, alerting layer, dashboard catalog, annotation system, and integration surface.

| Plane              | Responsibilities                                                   | Typical Failure                                         |
|--------------------|--------------------------------------------------------------------|---------------------------------------------------------|
| Presentation       | Panels, dashboards, variables, links, annotations, explore views   | Misleading units, hidden filters, unreadable density    |
| Data-source access | Connection, authentication, query proxying, health checks          | Expired credentials, TLS failure, wrong tenant or index |
| Governance         | Organizations, folders, teams, roles, provisioning, change control | Everyone is admin; dashboards drift from Git            |
| Reliability        | Database, sessions, plugins, HA, backups, self-monitoring          | SQLite on shared HA nodes; missing backup restore test  |
| Security           | Authentication, authorization, secrets, headers, public sharing    | Over-broad access or sensitive query leakage            |

### Deployment Decisions

- Single instance is suitable for a lab. Production may require external database storage, load balancing, consistent configuration, session handling, and controlled plugin deployment.
- Provision data sources and dashboards from version-controlled files or managed APIs when repeatability matters. Treat UI-only edits as an exception with an export-and-review workflow.
- Use separate folders and teams for platform, business-service, security, and shared executive views. Permissions should align with ownership and data sensitivity.
- Prefer service-account or workload credentials for data sources rather than personal user credentials. Rotate and scope them independently.
- Monitor Grafana itself: availability, HTTP errors, request duration, database health, alert evaluation, plugin failures, and data-source errors.

### Provisioning Example

#### Example: provisioned Grafana data sources

```
apiVersion: 1

datasources:
- name: prometheus-prod
  uid: prom-prod
  type: prometheus
  access: proxy
  url: http://prometheus:9090
  isDefault: true
  editable: false
  jsonData:
    timeInterval: 30s
- name: loki-prod
  uid: loki-prod
  type: loki
  access: proxy
  url: http://loki-gateway:3100
  editable: false
  jsonData:
```

```

derivedFields:
- name: TraceID
  matcherRegex: 'trace_id[=:]([a-f0-9]+)'
  datasourceUid: tempo-prod
  url: '$${__value.raw}'

```

**Provisioning Behavior** When a dashboard or data source is provisioned from files, the source of truth must be clear. UI edits can be overwritten by later provisioning updates.

## Production Readiness Checklist

- Externalize durable state and back it up according to recovery objectives.
- Use least-privilege authentication, team mappings, folder permissions, and service credentials.
- Provision critical data sources and dashboards from reviewed configuration.
- Pin approved plugins and test upgrades in a staging instance.
- Monitor Grafana availability, errors, query latency, alert evaluation, and database health.
- Document restore, rollback, credential rotation, and emergency access procedures.

## Troubleshooting Matrix

| Symptom                             | Likely Cause                                            | Verification                                                                 | Corrective Action                                                           |
|-------------------------------------|---------------------------------------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Dashboard loads but panels fail     | Data source unreachable or credentials invalid          | Run data-source health check; inspect Grafana logs and backend TLS/auth logs | Correct endpoint or credential; validate with a minimal query               |
| User sees unauthorized dashboards   | Folder/team mapping too broad                           | Review effective permissions and identity-provider group claims              | Reduce folder access; test with representative accounts                     |
| UI changes disappear                | Provisioned source overwrote database copy              | Inspect provisioning logs and dashboard UID/source metadata                  | Edit source-controlled JSON or change ownership model                       |
| Intermittent login/session failures | Inconsistent HA configuration or session/database issue | Compare node settings, cookies, time, database connectivity                  | Standardize configuration and repair shared state or load balancer behavior |

## Hands-On Lab

1. Deploy Grafana with Prometheus and Loki data sources in a disposable environment.
2. Create one admin, one editor, and one viewer team with separate folders.
3. Provision the two data sources and one dashboard from files.
4. Test a failed credential, unreachable backend, and dashboard rollback.
5. Export the configuration and record a restore rehearsal.

### Validation Evidence

- Sanitized provisioning files and environment map.
- Permission test matrix showing expected and denied access.
- Data-source health screenshots or API results for success and failure.
- Backup and restore rehearsal notes with measured recovery time.

## Interview Questions and Model Answers

### Q: Why should Grafana data sources use service credentials?

A: They decouple backend access from personal accounts, allow scoped permissions and independent rotation, and prevent dashboards from failing when an employee account changes.

### Q: What is the risk of making every user a Grafana admin?

A: Admins can alter data sources, plugins, organizations, alerting, and access controls. This destroys separation of duties and expands the impact of credential compromise or accidental changes.

**Q: What should be provisioned?**

A: Critical, repeatable resources such as data sources, folders, dashboards, alerting resources, and configuration should be managed as code when supported and appropriate.

**Chapter Summary**

- Grafana is a control and presentation layer over telemetry backends.
- Production operation requires explicit state, access, provisioning, backup, and self-monitoring decisions.
- Repeatability and least privilege matter more than convenience during initial setup.

# Chapter 22: Production Dashboard Engineering and Operational UX

*Designing dashboards that answer operator questions under pressure*

## Dashboard Hierarchy

A production dashboard should have an explicit audience and decision. The strongest designs follow a hierarchy: service overview, dependency or resource breakdown, and detailed diagnosis. A single page containing every metric usually slows response and hides the most important signal.

| Dashboard Type       | Primary Question                              | Recommended Content                                               |
|----------------------|-----------------------------------------------|-------------------------------------------------------------------|
| Executive or product | Are users and objectives healthy?             | SLIs, SLO status, error-budget trend, major business volume       |
| Service overview     | Is this service healthy now?                  | Traffic, errors, latency, saturation, deployments, active alerts  |
| Dependency           | Which downstream system is responsible?       | Per-dependency rate, error, latency, queue and timeout signals    |
| Resource             | Is infrastructure limiting the service?       | CPU, memory, disk, network, throttling, capacity and headroom     |
| Incident drill-down  | What changed and where should I inspect next? | Annotations, logs, exemplars, traces, runbook and ownership links |

## Panel Engineering Rules

- Start with user symptoms and service objectives. Do not lead with CPU unless CPU is the user-impact hypothesis.
- Choose correct units, decimals, thresholds, legends, time zones, and null handling. A wrong unit is a correctness defect.
- Use consistent color semantics across dashboards. Avoid treating normal variation as red or critical.
- Use rates for counters and show both numerator and denominator when a ratio can be misleading.
- Include ownership, environment, region, version, and data-freshness context without creating visual clutter.
- Use variables for bounded dimensions such as environment, cluster, namespace, service, or region. Avoid unbounded variables that issue expensive discovery queries.
- Link overview panels to Explore, logs, traces, deployments, and runbooks with the relevant time range and labels preserved.

## Dashboard Review Scorecard

| Dimension     | Pass Criteria                                              | Common Defect                                |
|---------------|------------------------------------------------------------|----------------------------------------------|
| Question      | Dashboard states the operational question and owner        | Generic title such as System Dashboard       |
| Correctness   | Queries, units, thresholds and aggregation are reviewed    | Averages hide tail latency or partial outage |
| Context       | Time range, environment, freshness and changes are visible | Viewer compares unlike environments          |
| Actionability | Links lead to logs, traces, runbooks or ownership          | Dead-end panel with no next step             |

| Dimension     | Pass Criteria                                 | Common Defect                                       |
|---------------|-----------------------------------------------|-----------------------------------------------------|
| Performance   | Queries are bounded and load time is measured | High-cardinality template variable scans all values |
| Accessibility | Readable contrast, legends and descriptions   | Meaning depends only on color                       |

### Example: queries for a service overview

```
# Useful dashboard query patterns

# Request rate by service
sum by (service) (rate(http_requests_total[5m]))

# Error ratio
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))

# p95 latency from a classic histogram
histogram_quantile(
  0.95,
  sum by (le, service) (rate(http_request_duration_seconds_bucket[5m]))
)
```

## Production Readiness Checklist

- Every dashboard has an owner, audience, purpose, review date, and source repository.
- Panels use reviewed units, transformations, thresholds, time ranges, and null behavior.
- Variables and links preserve environment and time context.
- Overview dashboards include user symptoms, dependencies, saturation, deployments, and data freshness.
- Load time and backend query cost are measured with representative time ranges.
- Dashboards are tested against normal, degraded, missing-data, and recovery scenarios.

## Troubleshooting Matrix

| Symptom                          | Likely Cause                                   | Verification                                              | Corrective Action                                                 |
|----------------------------------|------------------------------------------------|-----------------------------------------------------------|-------------------------------------------------------------------|
| Dashboard is slow                | Expensive query or unbounded variable          | Use query inspector and backend query metrics             | Add recording rules, reduce range/resolution, constrain variables |
| Panel shows no data after deploy | Label or data-source change                    | Compare query labels, annotations, and raw Explore output | Update query contract or restore compatible labels                |
| Ratio exceeds 100 percent        | Numerator/denominator dimensions do not match  | Inspect intermediate queries by labels                    | Align filters and aggregation; guard zero denominator             |
| Different dashboards disagree    | Different windows, aggregation or data sources | Compare query JSON and data-source UUIDs                  | Standardize definitions and reusable library panels               |

## Hands-On Lab

1. Create an overview dashboard for a synthetic checkout service.
2. Add RED metrics, saturation, deployment annotations, an active-alert list, and ownership links.
3. Create environment and service variables with bounded options.
4. Inject latency, errors, missing metrics, and a deployment change.
5. Review dashboard performance and perform a peer scorecard assessment.

### Validation Evidence

- Dashboard JSON in version control.
- Query review sheet with units and expected behavior.
- Screenshots for normal, degraded, missing-data, and recovered states.
- Measured dashboard load time and expensive-query findings.

## Interview Questions and Model Answers

### **Q: What makes a dashboard actionable?**

A: It answers a defined operational question, shows enough context to judge impact, and provides a clear drill-down path to evidence, ownership, and response guidance.

### **Q: Why are averages dangerous?**

A: Averages can hide regional, tenant, instance, or tail-latency failures. Operators should choose aggregations based on the failure mode and retain useful dimensions.

### **Q: What is dashboard-as-code?**

A: It is the practice of storing and reviewing dashboard definitions in version control so changes are repeatable, auditable, testable, and recoverable.

## Chapter Summary

- Dashboard quality depends on operational questions, not panel count.
- Correct queries, units, context, and drill-down links are reliability features.
- Normal, degraded, missing-data, and recovery testing is required.

## VOLUME 11C | CHAPTER 23

## Chapter 23: Grafana Alerting, Annotations, Correlations and Governance

*Connecting signals, changes, ownership and response without creating a second alerting mess*

### Unified Alerting Responsibilities

Grafana Alerting can evaluate queries and expressions across multiple data sources. This is useful when an alert requires data outside Prometheus or when teams standardize alert management in Grafana. However, mixed ownership between Prometheus rules, Loki rules, Grafana-managed rules, and backend-native alerts can create duplication. The organization must define which engine owns which alert classes.

| Capability          | Use                                                    | Governance Question                             |
|---------------------|--------------------------------------------------------|-------------------------------------------------|
| Alert rule          | Evaluate a condition on metrics or logs                | Which engine is authoritative?                  |
| Contact point       | Deliver a notification to a system                     | Who owns credentials and rotation?              |
| Notification policy | Route, group, mute and time notifications              | How are labels standardized?                    |
| Annotation          | Mark deployments, incidents or changes on time series  | Which events are trustworthy and retained?      |
| Correlation/link    | Move from a signal to logs, traces or external systems | Are labels and identifiers stable across tools? |

### Annotations and Change Intelligence

- Automatically annotate deployments, feature-flag changes, configuration rollouts, infrastructure maintenance, and incident start/end events.
- Include immutable identifiers such as commit SHA, deployment ID, change ticket, environment, service, and actor or automation identity.
- Do not treat every Git commit as a production change. Annotate the actual rollout or effective configuration change.
- Use trace IDs, request IDs, service names, and resource attributes consistently so derived fields and data links can connect metrics, logs, and traces.
- Test notification policies, mute timings, escalation labels, and missing-data handling with synthetic receivers before production use.

### Governance Model

| Control   | Minimum Standard                                               |
|-----------|----------------------------------------------------------------|
| Naming    | Stable folder, dashboard, rule group and label conventions     |
| Ownership | Team and service labels plus contact and escalation path       |
| Review    | Peer review for query, severity, duration, routing and runbook |
| Testing   | Synthetic state-transition, missing-data and recovery tests    |
| Audit     | Version history or source-control record and change rationale  |
| Lifecycle | Review date, stale-resource detection and deletion process     |

**Avoid Duplicate Paging** The same symptom should not page independently from Prometheus, Loki, Grafana and a cloud tool unless each notification has a distinct response contract.

## Production Readiness Checklist

- Define authoritative alert engines and prevent duplicate pages.
- Standardize service, team, environment, severity, region and runbook labels.
- Provision or version-control critical alerting resources where supported.
- Use synthetic receivers for route, grouping, mute and recovery tests.
- Create trusted deployment and change annotations.
- Link alerts and dashboards to logs, traces, ownership and runbooks.

## Troubleshooting Matrix

| Symptom                       | Likely Cause                                            | Verification                                          | Corrective Action                                    |
|-------------------------------|---------------------------------------------------------|-------------------------------------------------------|------------------------------------------------------|
| No notification               | Rule not evaluating, policy mismatch or contact failure | Inspect rule state, evaluation logs and policy labels | Fix query/policy/contact and replay synthetic test   |
| Duplicate pages               | Same symptom owned by multiple engines                  | Compare fingerprints, labels, routes and firing times | Choose one owner or make response contracts distinct |
| Annotation missing            | Deployment integration failed or timestamp wrong        | Inspect webhook/API response and event time           | Retry with idempotent event and correct UTC time     |
| Data link opens wrong context | Label mapping or URL variable mismatch                  | Inspect generated URL and source fields               | Normalize identifiers and encode variables correctly |

## Hands-On Lab

1. Create a Grafana-managed alert from a Prometheus query and one from a Loki query.
2. Route both to a synthetic contact point using standardized labels.
3. Add deployment annotations through an API or supported integration.
4. Create a dashboard data link from error metrics to matching logs.
5. Test pending, firing, recovery, missing-data, mute, and duplicate conditions.

### Validation Evidence

- Rule definitions and notification-policy export.
- Test transcript for each alert state and route.
- Deployment annotation with immutable change identifiers.
- Working metrics-to-logs link preserving service and time context.

## Interview Questions and Model Answers

### Q: When should an alert be evaluated in Grafana instead of Prometheus?

A: When the condition depends on non-Prometheus data or when the organization deliberately centralizes rule management there. The choice must still account for availability, ownership, version control, and failure modes.

### Q: What is the purpose of an annotation?

A: It overlays trusted change or event context on telemetry so operators can correlate behavior with deployments, configuration changes, incidents, and maintenance.

### Q: How do you prevent duplicate alerts?

A: Define authoritative engines, standardize labels and ownership, inventory rules across systems, and test that one symptom maps to one response contract.

## Chapter Summary

- Grafana can alert across multiple data sources, but ownership must be explicit.

- Annotations and stable identifiers accelerate correlation during incidents.
- Alert and dashboard governance requires naming, review, testing, audit, and lifecycle controls.

## Chapter 24: Centralized Logging Architecture and Structured Log Engineering

*Creating useful event records and reliable transport before selecting a storage engine*

### Logging Data Path

#### Reference architecture: centralized logging pipeline

```

Application / OS / container runtime
    |
    v
local stdout, file, journal or network endpoint
    |
    v
collector -> parse -> redact -> enrich -> buffer -> retry
    |
    v
transport / gateway / ingest pipeline
    |
    v
Loki, Elasticsearch or OpenSearch
    |
    v
Explore, dashboards, alerts and investigations
  
```

### Structured Log Contract

A log event is a schema-bearing operational record. Free-form text remains useful for a human-readable message, but stable fields enable filtering, aggregation, redaction, correlation, routing, and retention decisions.

| Field                  | Purpose                              | Example                         |
|------------------------|--------------------------------------|---------------------------------|
| @timestamp             | Event occurrence time in UTC         | 2026-08-01T06:10:13.421Z        |
| log.level              | Normalized severity                  | ERROR                           |
| message                | Human-readable event summary         | payment authorization timed out |
| service.name           | Owning service identity              | checkout-api                    |
| service.version        | Deployed version or build            | git-6fd91c2                     |
| deployment.environment | Environment                          | production                      |
| trace.id / trace_id    | Cross-signal correlation             | 4bf92f3577b34da6...             |
| event.action           | Stable event type                    | payment_authorization           |
| error.type             | Machine-readable error class         | TimeoutError                    |
| duration_ms            | Measured duration with explicit unit | 3002                            |

### Severity and Event Design

- DEBUG is detailed development or temporary diagnostic context and is normally disabled or sampled in production.
- INFO records meaningful lifecycle or business events without describing every successful low-level operation.
- WARN signals degraded or unexpected behavior that is recoverable and may require investigation if sustained.
- ERROR records a failed operation. It should describe impact and context, not merely repeat a stack trace.
- FATAL or CRITICAL indicates the process or essential function cannot continue safely.

- Log once at the boundary where the error becomes meaningful. Re-logging at every layer produces duplicate noise.
- Prefer stable event names and fields over parsing variable message text.

## Privacy and Cost Controls

| Risk                       | Preventive Control                                                    |
|----------------------------|-----------------------------------------------------------------------|
| Secrets and tokens         | Never log them; redact at source and collection; test known patterns  |
| Personal or regulated data | Minimize fields, classify data, restrict access, shorten retention    |
| Unbounded payloads         | Set size limits, truncate safely, store large objects elsewhere       |
| Duplicate events           | Single ownership and deduplication keys where appropriate             |
| Clock inconsistency        | UTC timestamps and synchronized hosts; preserve event and ingest time |
| Burst overload             | Local buffering, quotas, backpressure, sampling, and drop metrics     |

### Example: structured application event

```
{
  "@timestamp": "2026-08-01T06:10:13.421Z",
  "log.level": "ERROR",
  "message": "payment authorization timed out",
  "service.name": "checkout-api",
  "service.version": "git-6fd91c2",
  "deployment.environment": "production",
  "event.action": "payment_authorization",
  "trace.id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "error.type": "TimeoutError",
  "duration_ms": 3002,
  "payment.provider": "synthetic-bank"
}
```

## Production Readiness Checklist

- Publish and review a structured logging contract with required fields and severity rules.
- Prevent secrets and sensitive data at source; validate redaction with automated tests.
- Use UTC event timestamps and retain ingest timestamps when transport delay matters.
- Instrument collector buffers, retries, dropped records, parsing failures and delivery latency.
- Define ingest, retention, storage and query budgets per environment and data class.
- Provide correlation fields that align with tracing and service identity standards.

## Troubleshooting Matrix

| Symptom                | Likely Cause                                      | Verification                                                   | Corrective Action                                                      |
|------------------------|---------------------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------------|
| Logs arrive late       | Buffer backlog, downstream throttle or clock skew | Compare event time, ingest time, queue depth and retry metrics | Restore capacity, tune buffers, fix time synchronization               |
| Search fields missing  | Parser/schema mismatch                            | Inspect raw event and parse-failure counters                   | Fix producer schema or collector parser; replay safe sample            |
| Sensitive value found  | Source logged it or redaction failed              | Trace event path and pattern-test coverage                     | Stop exposure, rotate secret, delete under policy, add preventive test |
| Collector memory grows | Blocked output or oversized buffer                | Inspect queue, retries, output status and process memory       | Apply backpressure, disk buffer, limits or restore backend             |

## Hands-On Lab

1. Instrument a small service to emit JSON logs with required fields.
2. Create redaction tests for tokens, email-like values and authorization headers.
3. Collect logs from stdout and a file using a supported collector.
4. Simulate backend unavailability and observe buffering, retry and drop behavior.
5. Calculate daily ingest volume and retention storage for normal and burst traffic.

### Validation Evidence

- Logging contract and example schema.
- Redaction test report and sanitized failure example.
- Collector health metrics during outage and recovery.
- Ingest/retention capacity calculation with assumptions.

## Interview Questions and Model Answers

### **Q: Why is structured logging preferable to only free-form text?**

A: Stable fields support accurate filtering, aggregation, enrichment, redaction, routing, correlation, and schema evolution while retaining a readable message.

### **Q: Should every request be logged?**

A: Not automatically. The decision depends on diagnostic value, cost, privacy, sampling, and existing metrics/traces. High-volume success logs often need sampling or aggregation.

### **Q: Where should redaction happen?**

A: As early as possible, ideally before emission. Collector-side redaction is defense in depth, not permission to emit secrets.

## Chapter Summary

- Centralized logging begins with an event contract and reliable transport.
- Structured fields, timestamps, severity, correlation, privacy, and cost controls are core engineering concerns.
- Collector backpressure and delivery behavior must be observable and tested.

## Chapter 25: Grafana Loki Architecture, Storage, Tenancy and Operations

*Using label-indexed log streams without turning labels into an expensive full-text index*

### Loki Data Model

Loki indexes stream labels and stores compressed log content separately. A stream is defined by its complete label set. This makes label design a capacity and query-planning decision. Labels should identify stable, low-cardinality source dimensions; high-cardinality context belongs in structured metadata or parsed log fields.

| Appropriate Label    | Reason                           | Avoid as Label            |
|----------------------|----------------------------------|---------------------------|
| service_name         | Stable service grouping          | request_id                |
| environment          | Small controlled set             | user_id                   |
| cluster / region     | Bounded infrastructure dimension | full URL                  |
| namespace            | Useful Kubernetes boundary       | trace_id                  |
| container / workload | Operational source when bounded  | timestamp or random value |

### Deployment Modes

| Mode                        | Use                                      | Trade-off                                  |
|-----------------------------|------------------------------------------|--------------------------------------------|
| Monolithic / single binary  | Lab or small deployment                  | Simple but limited independent scaling     |
| Simple scalable             | Separated read, write and backend groups | Moderate operational complexity and scale  |
| Microservices / distributed | Large or specialized scale requirements  | Highest flexibility and operational burden |

### Core Operational Concerns

- Object storage is commonly used for durable chunks and index data in scalable deployments; configuration and compatibility must be validated for the chosen backend.
- The gateway, distributor, ingester, querier, query frontend/scheduler, compactor, index gateway, ruler, and cache layers have different scaling and failure characteristics.
- Retention and deletion require deliberate compactor/object-storage configuration and testing. A configured duration is not proven until expired data is actually removed.
- Multi-tenancy isolates requests by tenant identity. Authentication is normally provided by an authenticating reverse proxy or platform layer rather than assumed from an untrusted header.
- Query limits, ingestion limits, stream limits, per-tenant overrides, and fairness controls protect the service from accidental or abusive workloads.
- Meta-monitor Loki and its collectors using Prometheus metrics and operational logs, including query duration, bytes scanned, ingestion failures, rejected samples, chunk operations, cache behavior, and storage errors.

### Illustrative lab configuration - validate current schema and storage guidance

```
auth_enabled: true

server:
  http_listen_port: 3100

limits_config:
  retention_period: 168h
  ingestion_rate_mb: 4
  ingestion_burst_size_mb: 8
```

```

max_query_parallelism: 16
reject_old_samples: true
reject_old_samples_max_age: 168h

compactor:
  retention_enabled: true
  delete_request_store: filesystem

schema_config:
  configs:
    - from: 2026-01-01
      store: tsdb
      object_store: filesystem
      schema: v13
      index:
        prefix: loki_index_
        period: 24h

```

**Label Rule** A label set is a stream identity. Adding request IDs, trace IDs, user IDs, timestamps, or random values as labels creates stream explosion and poor performance.

## Production Readiness Checklist

- Choose deployment mode from measured ingest, query, retention and availability needs.
- Define a reviewed low-cardinality label taxonomy and reject uncontrolled labels.
- Protect tenant identity through trusted authentication and explicit authorization.
- Configure and test ingestion/query limits, fairness, retention and deletion.
- Monitor every major Loki component, storage backend, cache and gateway.
- Document backup, object-store protection, schema migration, upgrade and rollback procedures.

## Troubleshooting Matrix

| Symptom                     | Likely Cause                                   | Verification                                               | Corrective Action                                                        |
|-----------------------------|------------------------------------------------|------------------------------------------------------------|--------------------------------------------------------------------------|
| Ingestion returns 429       | Tenant ingestion or stream limit exceeded      | Inspect response, distributor metrics and tenant overrides | Reduce cardinality/rate or change approved limits                        |
| Queries scan huge volumes   | Selector too broad or labels insufficient      | Inspect query stats, bytes and chunks scanned              | Use selective stable labels and narrower time range                      |
| Old logs remain             | Retention/compactor/delete store misconfigured | Inspect compactor logs, object store and retention metrics | Correct retention path and validate deletion in lab                      |
| Tenant reads another tenant | Auth proxy or tenant header trust failure      | Test with separate credentials and inspect gateway headers | Enforce trusted identity mapping and reject user-supplied tenant headers |

## Hands-On Lab

1. Deploy a lab Loki instance with explicit retention and limits.
2. Create a good label set and a deliberately high-cardinality label set.
3. Measure streams, ingestion responses and query statistics for both.
4. Simulate storage or gateway failure, then observe recovery.
5. Validate tenant isolation and retention deletion using synthetic data.

### Validation Evidence

- Architecture diagram and component inventory.
- Label-cardinality comparison with stream counts and query statistics.
- Limit rejection evidence and corrected configuration.
- Tenant isolation and retention validation records.

## Interview Questions and Model Answers

**Q: How is Loki different from a traditional full-text index?**

A: Loki indexes labels that define streams and stores log content separately. Queries first select streams by labels and then filter or parse log lines.

**Q: Why is label cardinality critical?**

A: Each unique label set creates a stream. Unbounded values multiply streams, increase memory and index overhead, and make ingestion and queries less efficient.

**Q: What does multi-tenancy require beyond setting `auth_enabled`?**

A: A trusted authentication and authorization layer must map callers to tenant identity, protect headers, apply limits, and test isolation.

## Chapter Summary

- Loki economics and performance depend heavily on label design.
- Deployment mode, storage, retention, limits and tenancy must be validated as one system.
- Meta-monitoring is required to detect ingestion, query and storage failure.

## Chapter 26: LogQL Query Engineering, Metrics from Logs and Alerting

*Selecting streams first, then filtering, parsing, aggregating and measuring log-derived behavior*

### LogQL Query Pipeline

| Stage             | Purpose                  | Example                                           |
|-------------------|--------------------------|---------------------------------------------------|
| Stream selector   | Choose indexed streams   | {service_name="checkout-api", environment="prod"} |
| Line filter       | Reduce matching content  | = "timeout" or  ~ "5\d\d"                         |
| Parser            | Extract fields           | json,   logfmt,   regexp                          |
| Field filter      | Compare parsed values    | status >= 500   duration_ms > 1000                |
| Formatting        | Control displayed fields | line_format "{{.method}} {{.path}}"               |
| Range aggregation | Create metrics from logs | count_over_time(...[5m]), rate(...[5m])           |

### Query Examples

#### Example: LogQL log and metric queries

```
# Error logs for one service
{service_name="checkout-api", environment="prod"} |= "ERROR"

# JSON parsing and field filtering
{service_name="checkout-api"}
| json
| log_level = "ERROR"
| duration_ms > 1000

# Error event rate by service
sum by (service_name) (
  rate({environment="prod"} | json | log_level="ERROR" [5m])
)

# p95 duration extracted from logs
quantile_over_time(
  0.95,
  {service_name="checkout-api"}
  | json
  | unwrap duration_ms
  | __error__="" [5m]
)
```

### Correctness Rules

- Begin with the most selective stable stream selector possible. Content filters cannot recover the cost of selecting every stream for a long period.
- After parsing or unwrap operations, inspect and filter parser errors deliberately. Silent parse failures can bias log-derived metrics.
- Use explicit units. If duration is logged in milliseconds, do not label a panel as seconds without conversion.
- Log-derived metrics are useful when applications cannot expose metrics, but native metrics are usually more efficient and semantically stable for continuous measurement.
- Alert on sustained, actionable symptoms and include sample evidence, ownership, environment, and a runbook. Avoid paging on a single unclassified error log.
- Use recording rules or precomputed results only after validating the underlying query and expected label set.

## Query Review Checklist

| Question                                          | Reason                                            |
|---------------------------------------------------|---------------------------------------------------|
| Is the stream selector bounded?                   | Controls how much data must be searched           |
| Are parsed fields typed and consistently present? | Prevents incorrect comparisons or missing samples |
| Is <code>__error__</code> handled?                | Surfaces parser and unwrap failures               |
| Are labels preserved or aggregated intentionally? | Controls result cardinality and alert instances   |
| Is the time window aligned with response needs?   | Balances stability, delay and cost                |
| Can native metrics provide a better signal?       | Avoids unnecessary log query load                 |

## Production Readiness Checklist

- Queries use selective stream labels before content filters.
- Parser and unwrap errors are inspected and handled.
- Units, windows, grouping and result cardinality are reviewed.
- Expensive or repeated queries are measured and optimized.
- Log alerts have duration, severity, ownership, evidence and runbooks.
- Native metrics are preferred for stable continuous indicators when available.

## Troubleshooting Matrix

| Symptom                              | Likely Cause                                   | Verification                                          | Corrective Action                                                        |
|--------------------------------------|------------------------------------------------|-------------------------------------------------------|--------------------------------------------------------------------------|
| Query times out                      | Broad selector, long range or expensive parser | Inspect query statistics and frontend/querier metrics | Narrow labels/range, optimize parser, use recording rule                 |
| Metric query undercounts             | Parse failures or missing fields               | Query <code>__error__</code> and compare raw samples  | Fix schema/parser and handle invalid records                             |
| Alert creates thousands of instances | High-cardinality grouping                      | Inspect output labels and rule state                  | Aggregate to actionable service/team dimensions                          |
| Regex filter is slow                 | Complex expression across broad streams        | Measure bytes scanned and compare simpler filter      | Use labels, exact filters, parser fields or ingestion-time normalization |

## Hands-On Lab

1. Load synthetic JSON logs for two services and environments.
2. Write stream, line-filter, parser, field-filter and formatting queries.
3. Build error-rate and duration quantile queries from logs.
4. Inject malformed JSON and verify `__error__` handling.
5. Create and test one log-based alert with bounded labels.

### Validation Evidence

- Query workbook with expected and actual results.
- Query statistics before and after optimization.
- Parser-error evidence and corrected result.
- Alert state-transition test and runbook link.

## Interview Questions and Model Answers

**Q: What is the first operation in a LogQL query?**

A: A stream selector using indexed labels. It determines which log streams are candidates before line filtering and parsing.

**Q: Why filter \_\_error\_\_ after parsing?**

A: Parsing and unwrap stages can produce pipeline errors. Ignoring them can yield incomplete or misleading log-derived metrics.

**Q: When should logs become metrics?**

A: When event counts or extracted values provide a useful indicator and native instrumentation is unavailable or complementary. Stable high-volume indicators are usually better as native metrics.

## Chapter Summary

- LogQL combines label selection with line pipelines and metric aggregation.
- Query performance starts with selective stream labels and bounded time ranges.
- Parser errors, units, grouping and alert cardinality are correctness concerns.

# Chapter 27: Grafana Alloy Log Collection for Linux, Containers and Kubernetes

*Building supported, observable pipelines after Promtail end of life*

## Why Grafana Alloy

Grafana Alloy is an OpenTelemetry Collector distribution with native pipelines for Grafana telemetry systems and other compatible destinations. For Loki logging, Alloy components can discover files or Kubernetes workloads, read logs, process and relabel them, and write to Loki. Component health and pipeline behavior are visible through Alloy diagnostics and metrics.

**Migration Requirement** Promtail is end of life as of March 2, 2026. Existing Promtail configurations should be migrated to Alloy or another supported client, then tested for semantic equivalence.

## Pipeline Components

| Component Pattern                | Responsibility                                           |
|----------------------------------|----------------------------------------------------------|
| local.file_match                 | Discover files from path patterns                        |
| loki.source.file                 | Tail files and track positions                           |
| loki.source.journal              | Read systemd journal entries                             |
| discovery.kubernetes             | Discover Kubernetes resources                            |
| discovery.relabel / loki.relabel | Normalize targets or labels                              |
| loki.process                     | Parse, filter, redact, extract and transform log entries |
| loki.write                       | Batch and deliver entries to Loki endpoints              |

## Illustrative Alloy pipeline - validate current component syntax

```
logging {
  level = "info"
  format = "logfmt"
}

local.file_match "app_logs" {
  path_targets = [{
    __path__ = "/var/log/demo/*.log",
    service_name = "demo-app",
    environment = "lab",
  }]
}

loki.process "app_logs" {
  forward_to = [loki.write.default.receiver]

  stage.json {
    expressions = {
      level = "log.level",
      trace_id = "trace.id",
    }
  }

  stage.labels {
    values = { level = "" }
  }

  stage.replace {
    expression = "(?i)(authorization|token)=[^ ]+"
    replace = "${1}=REDACTED"
  }
}
```

```

    }
  }

  loki.source.file "app_logs" {
    targets      = local.file_match.app_logs.targets
    forward_to   = [loki.process.app_logs.receiver]
  }

  loki.write "default" {
    endpoint { url = "http://loki:3100/loki/api/v1/push" }
  }
}

```

## Collection Engineering

- Persist or understand position tracking so restarts do not silently duplicate or skip file content.
- Normalize Kubernetes labels to a small, stable taxonomy. Do not copy every pod annotation or label into Loki labels.
- Use processing stages for parsing, timestamp selection, redaction, sampling, dropping noise, and structured metadata. Test every stage with representative malformed input.
- Monitor bytes read, entries processed, parse failures, dropped entries, retries, batch behavior, delivery errors, file discovery, and pipeline health.
- Plan local buffering and failure behavior. A collector that blocks application stdout or exhausts node disk can turn an observability outage into an application outage.
- Use the Alloy conversion tools for Promtail migration, review warnings, and compare old/new output before cutover.

## Production Readiness Checklist

- Use supported Alloy components and pin reviewed configuration.
- Persist and monitor positions or checkpoints for file sources.
- Normalize labels and prevent Kubernetes metadata explosion.
- Test parse, timestamp, redaction, drop, sampling and malformed-record behavior.
- Monitor pipeline health, retries, drops, discovery and backend delivery.
- Perform staged migration with duplicate-run comparison and rollback.

## Troubleshooting Matrix

| Symptom                      | Likely Cause                                                | Verification                                                 | Corrective Action                                                  |
|------------------------------|-------------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------------------------|
| No new log lines             | File discovery, permissions or position issue               | Inspect Alloy UI/metrics, targets and file permissions       | Fix path/permissions; reset position only with duplicate-risk plan |
| Duplicate logs after restart | Checkpoint not persisted or two collectors read same source | Compare collector identity, positions and event fingerprints | Persist state and ensure single ownership                          |
| Loki rejects entries         | Bad labels, timestamp, tenant or rate limit                 | Inspect HTTP response and loki.write metrics                 | Correct labels/time/auth or approved limit                         |
| Node disk fills              | Buffer/output blockage or verbose local logs                | Inspect disk, queue, retries and backend health              | Restore output; cap buffers; rotate logs; apply backpressure       |

## Hands-On Lab

1. Install Alloy in a lab and expose its diagnostic interface and metrics.
2. Collect one JSON file, systemd journal entries, and Kubernetes pod logs or a simulated equivalent.
3. Add parsing, severity normalization, secret redaction, and stable labels.
4. Stop Loki, observe retries/failure behavior, then recover without uncontrolled duplication.
5. Convert a small Promtail configuration and compare output labels and content.

### Validation Evidence

- Alloy configuration and component graph screenshot.
- Pipeline health metrics and parse/drop counters.
- Redaction test proving the sensitive value never reaches Loki.

- Migration comparison showing equivalent source coverage and corrected label differences.

## Interview Questions and Model Answers

### **Q: What replaced Promtail for new Grafana logging deployments?**

A: Grafana recommends Grafana Alloy or another supported client. Promtail reached end of life in March 2026.

### **Q: Why are positions important for file tailing?**

A: They record how far the collector has read. Lost or shared state can cause duplicate ingestion or missed data after restart.

### **Q: What should be monitored on a log collector?**

A: Source discovery, bytes and entries read, parse failures, dropped records, buffers, retries, delivery errors, checkpoint health, resource use, and configuration status.

## Chapter Summary

- Grafana Alloy is the supported primary path for new Loki collection labs.
- Collection pipelines require state, label, parsing, redaction, buffering and failure-design decisions.
- Migration must compare behavior and preserve a rollback path.

## Chapter 28: Elasticsearch Logging: ECS, Data Streams, Ingest Pipelines and Lifecycle

*Designing searchable log data with controlled schemas, mappings, retention and shard economics*

### Elasticsearch Logging Model

Elasticsearch indexes document fields for search and aggregation. This provides powerful analytics but requires deliberate mappings, schemas, shard design, lifecycle management, ingest capacity, and access control. Continuously generated logs are commonly stored in data streams backed by hidden indices.

| Layer            | Responsibility                                             |
|------------------|------------------------------------------------------------|
| Producer / Agent | Emit or collect structured events                          |
| Ingest pipeline  | Parse, rename, convert, enrich, redact and route documents |
| Index template   | Define mappings, settings and data-stream behavior         |
| Data stream      | Stable name over rolling backing indices                   |
| Lifecycle        | Retention and storage management for backing indices       |
| Kibana / API     | Discover, query, visualize, alert and administer           |

### Elastic Common Schema

Elastic Common Schema provides standard field sets for events, services, hosts, containers, networks, errors, logs, and related domains. Adopting a common schema improves reuse and correlation, but teams should map fields intentionally rather than copying arbitrary payload keys into the root document.

| ECS Field                                | Use                             |
|------------------------------------------|---------------------------------|
| @timestamp                               | Event occurrence timestamp      |
| message                                  | Original human-readable message |
| log.level                                | Normalized severity             |
| service.name                             | Service identity                |
| event.category / type / action / outcome | Normalized event classification |
| error.type / message / stack_trace       | Error details                   |
| trace.id / transaction.id                | Correlation with tracing/APM    |
| data_stream.type / dataset / namespace   | Data-stream identity            |

### Ingest Pipeline Example

#### Example: ingest transformation and failure tagging

```
PUT _ingest/pipeline/logs-checkout-default
{
  "processors": [
    { "set": { "field": "service.name", "value": "checkout-api", "override": false }},
    { "rename": { "field": "level", "target_field": "log.level", "ignore_missing": true }},
```

```

{ "convert": { "field": "duration_ms", "type": "long", "ignore_missing": true }},
{ "remove": { "field": ["authorization", "token"], "ignore_missing": true }},
{ "set": { "field": "event.dataset", "value": "checkout.application" }}
},
"on_failure": [
{ "set": { "field": "error.message", "value": "{{ _ingest.on_failure_message }}" }},
{ "set": { "field": "tags", "value": ["ingest_failure"] }}
]
}

```

## Mapping and Shard Controls

- Use explicit mappings or controlled dynamic templates for important fields. Accidental field-type conflicts break queries and dashboards.
- Prevent mapping explosion from arbitrary JSON keys. Flatten, ignore, route, or limit uncontrolled objects.
- Size shards from measured ingest, retention, recovery and query behavior rather than creating one index per service per day by habit.
- Use data stream lifecycle or index lifecycle mechanisms appropriate to the deployment and current product guidance. Test rollover and deletion behavior.
- Dedicated ingest nodes may be appropriate for heavy transformation workloads. Measure processor latency, pipeline failures, rejected indexing, queue pressure and node resources.
- Protect APIs with TLS, authentication, role-based privileges, index or data-stream permissions, and audited administrative access.

### Example: pipeline simulation

```

# Simulate an ingest pipeline before sending production data
POST _ingest/pipeline/logs-checkout-default/_simulate
{
  "docs": [
    { "_source": {
      "@timestamp": "2026-08-01T06:10:13Z",
      "level": "ERROR",
      "message": "synthetic timeout",
      "duration_ms": "3002",
      "token": "must-not-be-indexed"
    }}
  ]
}

```

## Production Readiness Checklist

- Use a documented schema and reviewed ECS mapping strategy.
- Create data streams/templates with controlled mappings and settings.
- Simulate ingest pipelines and test on\_failure handling before rollout.
- Limit dynamic fields and monitor mapping growth, rejected documents and field conflicts.
- Design lifecycle, shard size, replica, recovery and retention from measured workload needs.
- Apply TLS, least-privilege roles, credential rotation, audit and backup/restore tests.

## Troubleshooting Matrix

| Symptom                      | Likely Cause                                     | Verification                                        | Corrective Action                                                     |
|------------------------------|--------------------------------------------------|-----------------------------------------------------|-----------------------------------------------------------------------|
| Documents rejected           | Mapping conflict or pipeline processor failure   | Inspect bulk response and ingest failure fields     | Correct mapping/pipeline; reprocess sanitized failed records          |
| Cluster has many tiny shards | Over-segmented index strategy                    | Inspect shard count, size and creation pattern      | Consolidate data streams and adjust rollover criteria                 |
| Search is slow               | Expensive query, field type or shard layout      | Use query profiling and node/search metrics         | Fix mapping/query, reduce shards, add capacity or routing             |
| Disk watermark triggered     | Retention lag, replica pressure or ingest growth | Inspect allocation, disk, lifecycle and ingest rate | Restore capacity, enforce retention, reduce ingest or replicas safely |

## Hands-On Lab

1. Create a composable index template and a logs data stream in a lab.

2. Create an ingest pipeline that maps fields to ECS and removes sensitive values.
3. Use the simulate API with valid, malformed and conflicting documents.
4. Load synthetic data, inspect mappings and run Discover queries.
5. Configure a short lab lifecycle and validate rollover/retention behavior.

#### Validation Evidence

- Template, pipeline and lifecycle definitions.
- Simulation output for success and on\_failure paths.
- Mapping inventory proving sensitive fields are absent.
- Shard/retention measurements and deletion validation.

## Interview Questions and Model Answers

### Q: What is an Elasticsearch data stream?

A: It is a named resource for append-oriented time-series data backed by a sequence of managed indices, enabling rollover and lifecycle behavior behind a stable name.

### Q: Why use ECS?

A: A common schema improves field consistency, reusable searches, dashboards, correlations and integrations across diverse event sources.

### Q: What is mapping explosion?

A: Uncontrolled dynamic fields create excessive mappings, memory/cluster-state overhead and query complexity. It is prevented with schema controls, templates and field limits.

## Chapter Summary

- Elasticsearch offers powerful field indexing but demands schema, mapping and shard discipline.
- Data streams, ingest pipelines, templates and lifecycle controls form one operational design.
- Simulation, failure handling, security and recovery tests are production requirements.

## VOLUME 11C | CHAPTER 29

## Chapter 29: OpenSearch Logging: Data Prepper, PPL, ISM, Security and Operations

*Building an open-source search and analytics logging platform with explicit lifecycle and tenancy controls*

### OpenSearch Observability Stack

OpenSearch provides a search and analytics engine, OpenSearch Dashboards, security controls, index management, alerting, and observability experiences. Data Prepper can receive, transform, buffer, route, and deliver logs or other telemetry to OpenSearch. Product capabilities and plugins vary by distribution, so the deployed feature set must be inventoried and tested.

| Component                | Role                                                                  |
|--------------------------|-----------------------------------------------------------------------|
| OpenSearch nodes         | Index, store, search and aggregate documents                          |
| OpenSearch Dashboards    | Discover, PPL, visualizations, administration and observability views |
| Data Prepper             | Pipeline-based ingest, transformation, routing and buffering          |
| Index templates/mappings | Schema and settings for matching indices or data streams              |
| Index State Management   | Policy-driven transitions and actions for indices                     |
| Security plugin          | Authentication, roles, tenants, certificates and audit options        |

### Data Prepper Pipeline Pattern

#### Illustrative Data Prepper pipeline - validate current plugin options

```
log-pipeline:
  source:
    http:
      path: /logs
      ssl: false
  processor:
    - parse_json:
        source: message
    - add_entries:
        entries:
          - key: service.name
            value: checkout-api
    - delete_entries:
        with_keys: [authorization, token]
  sink:
    - opensearch:
        hosts: ["https://opensearch:9200"]
        index: logs-checkout-%{yyyy.MM.dd}
        username: ${OPENSEARCH_USERNAME}
        password: ${OPENSEARCH_PASSWORD}
        insecure: false
```

### Piped Processing Language

#### Example: OpenSearch PPL investigations

```
source = logs-checkout-*
| where `service.name` = 'checkout-api'
| where `log.level` = 'ERROR'
| stats count() as errors by `error.type`
| sort - errors

source = logs-checkout-*
| where duration_ms > 1000
| stats avg(duration_ms), percentile(duration_ms, 95) by `service.name`
```

## Index State Management

Index State Management policies can perform actions and transitions based on age, size, or other conditions. A common lifecycle moves data through hot, warm, read-only, snapshot, and deletion states where the infrastructure supports those tiers. Policies must be attached correctly and their jobs monitored; defining JSON alone does not prove lifecycle execution.

| Control            | Operational Requirement                                       |
|--------------------|---------------------------------------------------------------|
| ISM policy         | Reviewed states, transitions, actions and deletion safeguards |
| Security roles     | Index/data-stream permissions and tenant boundaries           |
| Certificates       | Trusted transport and HTTP TLS with rotation plan             |
| Snapshots          | Repository protection and tested restore                      |
| Cluster allocation | Shard placement, replica health and zone awareness            |
| Ingest protection  | Bulk rejection monitoring, queues, backpressure and limits    |

## Production Readiness Checklist

- Inventory installed OpenSearch plugins and supported versions/features.
- Use explicit mappings/templates and a controlled field schema.
- Secure HTTP and transport traffic, roles, tenants and administrative actions.
- Configure and validate Data Prepper buffering, retries, dead-letter or failure handling.
- Attach and monitor ISM policies, then prove transitions and deletion.
- Test snapshots, restore, shard recovery, node failure and certificate rotation.

## Troubleshooting Matrix

| Symptom                            | Likely Cause                                     | Verification                                              | Corrective Action                                            |
|------------------------------------|--------------------------------------------------|-----------------------------------------------------------|--------------------------------------------------------------|
| Data Prepper pipeline stops        | Source/sink auth, certificate or processor error | Inspect pipeline logs, metrics and sink response          | Correct secret/TLS/schema; replay synthetic data             |
| ISM policy not applied             | Template mismatch or policy not attached         | Inspect policy explain/status and index settings          | Attach policy correctly and verify next execution            |
| Yellow/red cluster                 | Unassigned primary/replica shards                | Use cluster allocation explanation and node health        | Restore node/capacity or correct allocation/replica settings |
| User can access wrong tenant/index | Role or tenant mapping too broad                 | Test effective permissions with representative identities | Correct backend roles, index patterns and tenant permissions |

## Hands-On Lab

1. Deploy a small OpenSearch and Dashboards lab with TLS or a clearly isolated local setup.
2. Create a Data Prepper pipeline for synthetic JSON logs.
3. Define mappings and an ISM policy with a short lab transition.
4. Use PPL to investigate errors and latency.
5. Test role isolation, node or sink failure, and snapshot restore.

### Validation Evidence

- Component/plugin inventory and architecture diagram.
- Data Prepper success/failure metrics and sanitized logs.
- PPL query workbook and dashboard evidence.
- ISM transition, access-denial and snapshot-restore records.

## Interview Questions and Model Answers

### **Q: What is Data Prepper?**

A: It is a pipeline-oriented data collector and processor commonly used with OpenSearch to receive, transform, buffer, route and deliver observability data.

### **Q: What is ISM?**

A: Index State Management applies policies containing states, actions and transitions to indices so lifecycle operations such as rollover, read-only, snapshot or deletion can be automated.

### **Q: How do OpenSearch tenants differ from index permissions?**

A: Tenants commonly separate saved objects and dashboard content, while index permissions govern which underlying data a role may read or modify. Both must be configured correctly.

## Chapter Summary

- OpenSearch logging combines search, ingest, lifecycle, dashboards and security components.
- Data Prepper and ISM require observable failure handling and execution validation.
- Plugin inventory, tenancy, snapshots and shard recovery are core operational concerns.

## VOLUME 11C | CHAPTER 30

## Chapter 30: Unified Observability and Centralized Logging Platform Capstone

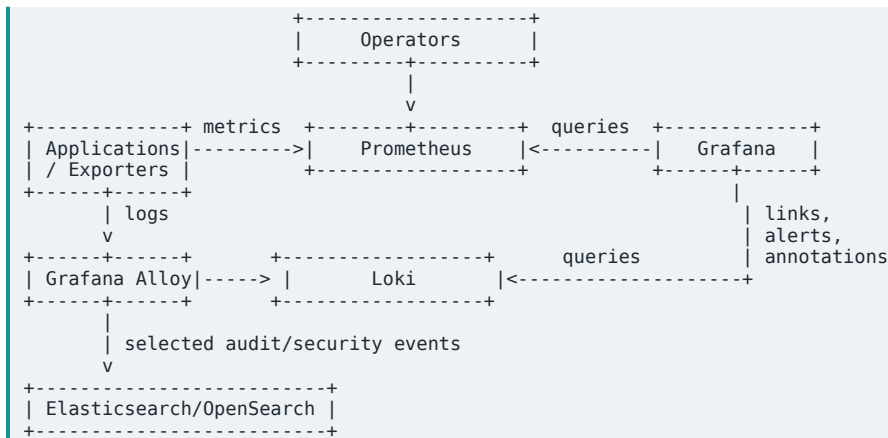
*Correlating metrics, logs, alerts and changes through controlled failures and recovery*

### Capstone Scenario

Build a small production-style observability platform for a synthetic checkout system containing an API, payment worker, database dependency, and reverse proxy. Prometheus provides metrics, Loki receives application/container logs through Grafana Alloy, and Grafana provides dashboards, Explore, annotations, data links, and alert views. Add either Elasticsearch or OpenSearch as a second log analytics path for selected security or audit events so that storage choices are justified by workload rather than fashion.

### Reference Architecture

#### Capstone architecture



### Required Deliverables

| Deliverable                | Acceptance Standard                                                                 |
|----------------------------|-------------------------------------------------------------------------------------|
| Architecture and ownership | Components, data paths, trust boundaries, owners and dependencies documented        |
| Telemetry contracts        | Metric names/labels and structured log schema reviewed                              |
| Grafana provisioning       | Data sources, folders and dashboards reproducible from code                         |
| Loki pipeline              | Alloy collection, redaction, labels, retention and limits validated                 |
| Search platform path       | Selected event class, schema, ingest pipeline, lifecycle and access justified       |
| Correlation                | Metrics-to-logs links, deployment annotations and trace/request IDs where available |
| Alerting                   | Actionable alerts with routing, runbooks and state-transition tests                 |
| Operations                 | Backups, restore, upgrade, rollback, capacity and incident procedures               |
| Evidence                   | Sanitized commands, screenshots, metrics, logs, timings and failure records         |

## Failure Drill Catalogue

| Drill                          | Expected Observation                                      | Recovery Proof                                                        |
|--------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------|
| Application latency injection  | RED dashboard and matching slow logs                      | Latency returns; alert resolves; event timeline retained              |
| Backend authentication failure | Grafana panel/data-source error and backend auth evidence | Credential repaired/rotated and health check passes                   |
| Loki unavailable               | Alloy retries/errors and dashboard log gap                | Delivery resumes within defined behavior; no uncontrolled node impact |
| High-cardinality label         | Stream growth and rejection/cost evidence                 | Label removed; pipeline guardrail prevents recurrence                 |
| Malformed JSON burst           | Parser errors and quarantined/dropped records             | Valid logs continue; failure rate returns to normal                   |
| Search-node loss               | Shard health and degraded query behavior                  | Allocation recovers and data remains available per replica design     |
| Lifecycle misconfiguration     | Retention does not execute or data risk identified        | Policy corrected and lab deletion/restore validated                   |
| Grafana rollback               | Bad dashboard/provisioning change detected                | Known-good version restored and audit record created                  |

## Implementation Sequence

1. Create a service inventory, data classification, ownership map, SLOs, and incident response expectations.
2. Deploy the synthetic services and instrument native Prometheus metrics plus structured JSON logs.
3. Deploy Prometheus and validated rules from Volume 11B.
4. Deploy Loki and Grafana Alloy with controlled labels, redaction, retention and limits.
5. Deploy Grafana with provisioned data sources, folders, dashboards, annotations and data links.
6. Choose Elasticsearch or OpenSearch for a justified selected event class, then configure schema, ingest, lifecycle and access.
7. Create alerts, runbooks and notification tests without using production paging channels.
8. Execute all failure drills and record detection, diagnosis, mitigation and recovery timings.
9. Perform backup/restore and configuration rollback rehearsals.
10. Prepare a portfolio evidence package and a fifteen-minute architecture defense.

## Acceptance Gates

- No sensitive test token appears in either centralized logging backend.
- Overview dashboard identifies service impact and links to matching logs within the same time window.
- Deployment annotations identify the exact version and rollout event.
- Collector/backend outages do not exhaust application or node resources.
- High-cardinality labels are detected and blocked or corrected.
- Lifecycle behavior is proven using disposable data; backups and restore are proven separately.
- Every alert has owner, severity, duration, evidence, routing and runbook.
- Configuration rollback restores a known-good state without manual reconstruction.
- Capacity calculations include daily ingest, retention, replicas, compression assumption, burst factor and headroom.
- All evidence is sanitized and reproducible.

## Production Readiness Checklist

- Architecture, ownership, trust boundaries and recovery objectives are approved.
- All configuration is versioned, reviewed, validated and rollback-capable.

- Dashboards and alerts are tested against normal, degraded, missing-data and recovery states.
- Privacy, access, retention, ingest budgets and query limits are enforced.
- Platform and collector self-monitoring covers every data path.
- Failure drills, restore and architecture defense meet the acceptance gates.

## Troubleshooting Matrix

| Symptom                        | Likely Cause                                   | Verification                                                  | Corrective Action                                          |
|--------------------------------|------------------------------------------------|---------------------------------------------------------------|------------------------------------------------------------|
| Metrics healthy but users fail | Missing business/SLO signal or dependency logs | Compare synthetic checks, RED signals and dependency evidence | Add user-centric instrumentation and dependency visibility |
| Logs missing during incident   | Collector/backend failure or label mismatch    | Inspect Alloy/Loki health and broad source selector           | Recover pipeline; fix labels; document blind spot          |
| Dashboard points to wrong logs | Data-link mapping or time context wrong        | Inspect generated URL and labels                              | Normalize service IDs and preserve time range              |
| Recovery not trusted           | No measured restore or rollback evidence       | Run controlled restore/rollback rehearsal                     | Record RTO/RPO, gaps and corrected procedure               |

## Hands-On Lab

1. Implement the reference architecture using disposable infrastructure.
2. Run the complete failure drill catalogue.
3. Measure detection, diagnosis, mitigation and recovery for each drill.
4. Review costs, privacy, access, retention and platform capacity.
5. Present the architecture and evidence to a peer reviewer.

### Validation Evidence

- Git repository containing sanitized configuration and runbooks.
- Architecture, ownership and data-classification diagrams.
- Failure-drill report with timings and screenshots.
- Backup/restore, lifecycle and rollback evidence.
- Capacity and cost workbook with assumptions.
- Architecture-defense notes and reviewer findings.

## Interview Questions and Model Answers

### Q: How do you choose between Loki and Elasticsearch/OpenSearch?

A: Choose from query patterns, schema needs, full-text requirements, ingest economics, operational skills, retention, compliance and ecosystem integrations. Loki is optimized around label-selected streams; search engines index fields more broadly and require mapping/shard discipline.

### Q: What proves an observability platform is production-ready?

A: Controlled failure and recovery evidence, tested access/privacy/retention, predictable capacity, self-monitoring, reproducible configuration, actionable dashboards/alerts, and documented ownership and rollback.

### Q: What is the most important capstone outcome?

A: The ability to detect user impact, correlate signals, identify a plausible cause, execute a safe response, and prove recovery using reproducible evidence.

## Chapter Summary

- The capstone integrates metrics, logs, dashboards, changes, alerts, security, lifecycle and recovery.
- Tool choice must be justified by workload and operating model.
- Production readiness is demonstrated through controlled evidence, not configuration screenshots alone.

# Grafana and Logging Command Reference

```
# Grafana health
curl -fsS http://localhost:3000/api/health

# Loki readiness and labels
curl -fsS http://localhost:3100/ready
curl -fsS http://localhost:3100/loki/api/v1/labels

# Query Loki API (URL-encode production queries)
curl -G -s http://localhost:3100/loki/api/v1/query_range \
  --data-urlencode 'query={service_name="checkout-api"}' \
  --data-urlencode 'limit=50'

# Elasticsearch cluster and data streams
curl -s -u "$ES_USER:$ES_PASS" "$ES_URL/_cluster/health?pretty"
curl -s -u "$ES_USER:$ES_PASS" "$ES_URL/_data_stream?pretty"

# OpenSearch cluster health and shard view
curl -s -u "$OS_USER:$OS_PASS" "$OS_URL/_cluster/health?pretty"
curl -s -u "$OS_USER:$OS_PASS" "$OS_URL/_cat/shards?v"
```

## Volume 11C Completion Checklist

- I can explain Grafana state, data-source, permission, provisioning and HA boundaries.
- I can build dashboards that answer user-impact and diagnosis questions with correct units and links.
- I can govern Grafana alerting, annotations, correlations and change lifecycle.
- I can define a structured logging contract with privacy, timestamp, severity and correlation controls.
- I can explain collector buffering, retries, checkpoints, drops, backpressure and outage behavior.
- I can deploy and operate Loki with bounded labels, retention, limits, tenancy and meta-monitoring.
- I can write LogQL log and metric queries and handle parser errors correctly.
- I can collect logs with Grafana Alloy and migrate a small Promtail configuration safely.
- I can design Elasticsearch data streams, ECS mappings, ingest pipelines and lifecycle controls.
- I can design OpenSearch pipelines, PPL investigations, ISM policies, security and restore tests.
- I can correlate metrics, logs, deployments and runbooks in a unified incident workflow.
- I can present sanitized evidence and defend trade-offs in an interview.

## Ten Additional Practice Projects

### 1. Dashboard Quality Audit

Review ten dashboards for question, correctness, context, performance, ownership, accessibility and drill-down paths.

### 2. Provisioning Repository

Create a Git-managed Grafana folder, data-source and dashboard structure with validation and rollback.

### 3. Logging Contract Linter

Validate required JSON fields, field types, severity values, forbidden secrets and maximum event size.

### 4. Alloy Failure Laboratory

Test file rotation, checkpoint loss, backend outage, parse failure, duplicate collectors and disk pressure.

### 5. Loki Cardinality Clinic

Measure stream growth from good and bad labels, then implement relabel/drop guardrails.

### 6. LogQL Workbook

Create thirty queries covering filters, parsers, unwrap, aggregations, parser errors and alerts.

## 7. ECS Mapping Project

Map a custom application schema to ECS and document non-ECS extension fields.

## 8. Lifecycle Validation Harness

Generate disposable data and verify data-stream lifecycle or ISM transitions and deletion.

## 9. Access Isolation Test

Test Grafana folders, Loki tenants and search-backend indices/tenants with representative roles.

## 10. Incident Correlation Exercise

Link an SLO alert to dashboard, logs, deployment annotation, trace ID and runbook in one workflow.

# 30-Day Practice Plan

| Period     | Focus                                                                                           |
|------------|-------------------------------------------------------------------------------------------------|
| Days 1-3   | Deploy Grafana, provision data sources, configure users/teams/folders, and test backup/restore. |
| Days 4-6   | Build overview and drill-down dashboards; review units, variables, links and query cost.        |
| Days 7-9   | Create annotations, correlations and Grafana-managed alert tests.                               |
| Days 10-12 | Implement a structured logging contract, redaction tests and ingest capacity estimate.          |
| Days 13-15 | Deploy Loki; test labels, limits, retention, tenancy and component monitoring.                  |
| Days 16-18 | Practice LogQL filters, parsers, unwrap, aggregations, errors and alerts.                       |
| Days 19-21 | Deploy Grafana Alloy; test files, journal/container sources, outages and migration.             |
| Days 22-25 | Build an Elasticsearch data stream, ECS mapping, ingest pipeline and lifecycle lab.             |
| Days 26-27 | Build an OpenSearch/Data Prepper/PPL/ISM lab and security tests.                                |
| Days 28-30 | Complete capstone, failure drills, restore, evidence package and interview review.              |

## Glossary

| Term          | Definition                                                                               |
|---------------|------------------------------------------------------------------------------------------|
| Annotation    | Time-stamped change or event context displayed with telemetry.                           |
| Data source   | Grafana connection and query configuration for a telemetry backend.                      |
| Data stream   | Stable named resource over rolling backing indices for append-oriented time-series data. |
| Derived field | Parsed log field that can generate a link to another data source or external system.     |

| Term                | Definition                                                                         |
|---------------------|------------------------------------------------------------------------------------|
| ECS                 | Elastic Common Schema, a standard set of fields for events and observability data. |
| Ingest pipeline     | Ordered processors that transform documents before indexing.                       |
| ISM                 | OpenSearch Index State Management for policy-driven index lifecycle actions.       |
| Label               | Loki indexed key-value pair that contributes to a stream identity.                 |
| LogQL               | Loki query language for stream selection, line processing and log-derived metrics. |
| Mapping             | Search-engine definition of field types and indexing behavior.                     |
| PPL                 | OpenSearch Piped Processing Language for chained search and analysis commands.     |
| Provisioning        | Creating Grafana resources from files or APIs rather than manual UI steps.         |
| Structured metadata | Loki metadata attached to entries without becoming stream labels.                  |
| Tenant              | Logical isolation boundary for data or saved resources.                            |
| Trace ID            | Identifier used to correlate an operation across services and telemetry signals.   |

## Official Documentation Index

- Grafana introduction: <https://grafana.com/docs/grafana/latest/introduction/>
- Grafana dashboards: <https://grafana.com/docs/grafana/latest/visualizations/dashboards/>
- Grafana dashboard best practices: <https://grafana.com/docs/grafana/latest/visualizations/dashboards/build-dashboards/best-practices/>
- Grafana provisioning: <https://grafana.com/docs/grafana/latest/administration/provisioning/>
- Grafana Alerting: <https://grafana.com/docs/grafana/latest/alerting/>
- Grafana Loki documentation: <https://grafana.com/docs/loki/latest/>
- Loki labels: <https://grafana.com/docs/loki/latest/get-started/labels/>
- Loki operations: <https://grafana.com/docs/loki/latest/operations/>
- Loki meta-monitoring: <https://grafana.com/docs/loki/latest/operations/meta-monitoring/>
- Grafana Alloy documentation: <https://grafana.com/docs/alloy/latest/>
- Send logs to Loki with Alloy: <https://grafana.com/docs/alloy/latest/tutorials/send-logs-to-loki/>
- Migrate Promtail to Alloy: <https://grafana.com/docs/alloy/latest/set-up/migrate/from-promtail/>
- Elasticsearch reference: <https://www.elastic.co/guide/en/elasticsearch/reference/current/>
- Elasticsearch ingest pipelines: <https://www.elastic.co/guide/en/elasticsearch/reference/current/ingest.html>
- Elasticsearch data stream lifecycle: <https://www.elastic.co/guide/en/elasticsearch/reference/current/data-stream-lifecycle.html>
- Elastic Common Schema field reference: <https://www.elastic.co/docs/reference/ecs/ecs-field-reference>
- OpenSearch documentation: <https://docs.opensearch.org/latest/>
- OpenSearch observability: <https://docs.opensearch.org/latest/observing-your-data/>
- OpenSearch index management: <https://docs.opensearch.org/latest/dashboards/im-dashboards/index-management>
- OpenSearch log analysis: <https://docs.opensearch.org/latest/observing-your-data/exploring-observability-data/discover-logs/>

**Version Note** Grafana, Loki, Alloy, Elastic and OpenSearch evolve rapidly. Verify stable releases, schemas, plugin support, licensing, migration guidance and security recommendations in official documentation before production implementation.

# VOLUME 11C COMPLETE

Grafana, Centralized Logging, Loki, Elastic and OpenSearch

**Next Part** Volume 11D - OpenTelemetry, Distributed Tracing, APM, Profiling and eBPF.